

Juan G. Villegas

MAT 342 – Explorations in Data Science

HW 03: CART Models

Questions:

1. How does CART decide how to split the data and thus build the decision tree?

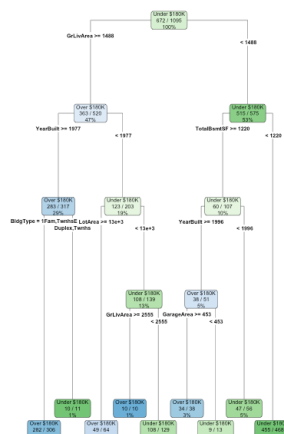
The cart model starts off with a root node, which is the target variable. After this, it creates a branch that splits the data almost in half. The cart model continues to split into two until there is a good amount of leaf nodes which convey a picture of our data. CART splits the data enough times to show a good balance of complexity & accuracy.

Using the example from your video, the cart model had a target variable of 'people that made <\$50k a year'. After that the CART Model split the data by categorizing people by their marriage (married or not married) because it splits the data *almost evenly (53% & 47%). For the people that were not married, the model decided to not further split the data because a high percentage of unmarried people (93.61%) made less than <\$50k a year. For the people that where married, CART decided to split the data again and added education as a variable. It split the data into 2 branches again: people that had an Associates/Bachelors & everything else (GED, HS grad, <9th, & post grad). CART left it at those child nodes because it showed that people with 'everything else' & where married had a 68% chance of having less than <\$50k in our data set while, people who were married & had either an associates or bachelor's degree had a 70% chance of having >\$50k income in the data set.

2. Is CART a prediction, estimation, or classification model?

A classification of data science method

4. Plot the model:

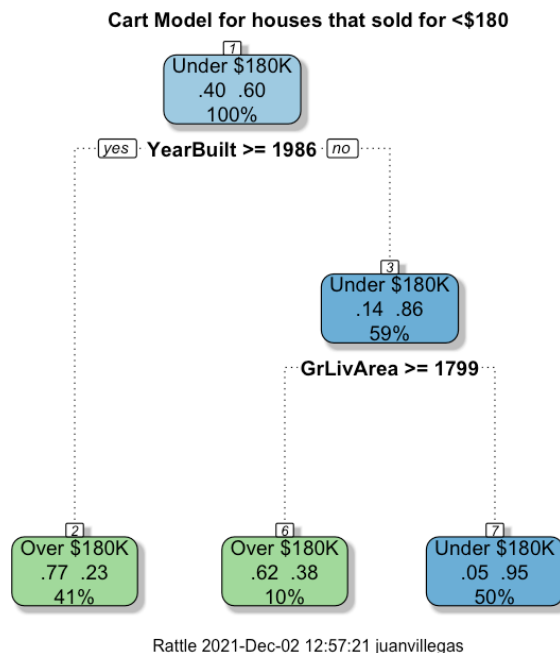


- Obtain the accuracy of the model based on the Training data and based on the Test data.
Compare the measures of accuracy to discuss the model's performance

Comparing our training data set accuracy against its own data. We got that our model is about 91% accurate when tested against its own data.

When our training data set accuracy is tested against the test data (or data different than itself). We got that our model is about 90% accurate when tested against a dataset other than itself.

- Now use the *trainControl()* and *train()* commands to make a CART model using 10-fold cross-validation. Plot the model.



- Obtain the accuracy of the model based on the Training data and based on the Test data.
Compare the measures of accuracy to discuss the model's performance.

The accuracy of the model based on the training data was 83% and the accuracy based on the testing data was 82%. The measures of accuracy for both models are very* similar with the training data performing slightly better by 1%.

- Compare the accuracy of the 10-fold cross validated model to the previous CART model. Consider the complexity of the model (simpler is better) and the accuracy (higher is better) to choose which model you would move forward with.

The accuracy of the 10-fold cross validation model is a little less than 10% accurate than the previous CART model. I would choose the less complex, 10-fold cross valid model, over the very complex one because it is a lot straighter to the point. I am giving a to the point explanation for houses that sold for less than 180k that is pretty accurate.

9. Report the decision rules from the 10-fold cross validated model, along with the accuracy of each rule and the percent of the data the rule applies to.

Our Target Variable for our CART model was, "Houses that sold for less than \$180k". 60% of the houses in our data set sold for less than \$180k, while 40% of the houses sold for more or equal to \$180k.

Our model has 3 leaf nodes and 2 branches. The first branch splits our data almost in half by the year that it was built.

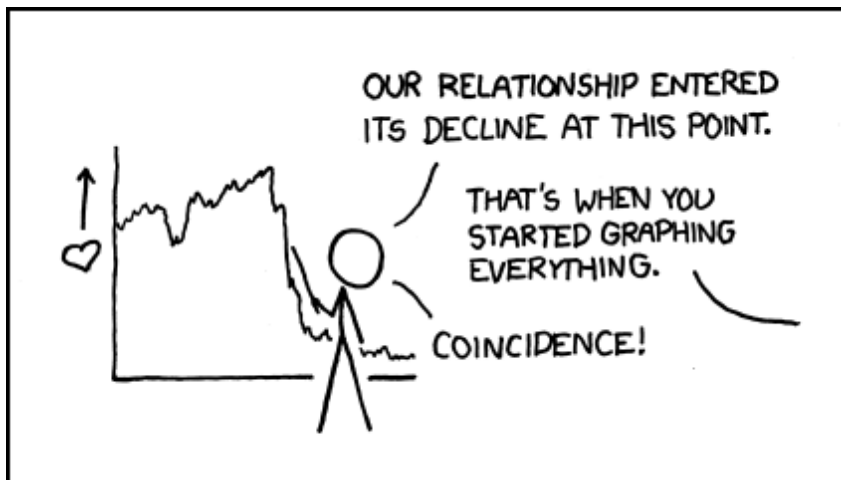
If the house was built on or after 1986, which is 41% of the houses in our data set, then 77% of those houses sold for more or equal to \$180k.

If the house was built before 1986, which is 59% of our data, then the model split the data again by Ground Floor Living Area being more or equal to 1799 or less than 1799.

If the Ground Floor Living Area is more or equal to 1799, which is 10% the houses in the data set, then 62% of those houses sold for more than \$180k.

If the Ground Floor Living Area is less than 1799, which is 50% of the houses in our data set, then 95% of them sold for less than \$180k

10. What has been your favorite picture from the note packets so far?



Appendix

Question 3: Run the *HW03_prelude* code from Blackboard, which will create the categorical target variable of *Sale Price Over \$180K or Not (over180)*, subset the data, and make the categorical variables into factors. Then partition the data into 75% Training and 25% Test data and validate the partition on two variables – one categorical and one numeric.

```
> #Partitioning the Data Numerically
> set.seed(69)
> InTrain <- createDataPartition(y = subhouse$over180, p = .75, list = FALSE)
> Train_Data <- subhouse[InTrain,]
> Test_Data <- subhouse[-InTrain,]
> Train_Data$trainortest <- rep("train", nrow(Train_Data))
> Test_Data$trainortest <- rep("test", nrow(Test_Data))
> All_Data <- rbind(Train_Data, Test_Data)
> boxplot(LotArea ~ as.factor(trainortest), data = All_Data)
> kruskal.test(LotArea ~ as.factor(trainortest), data = All_Data)$p.value
[1] 0.2314099
```

```
> #Partitioning for Categorical Data
> #Use the 'All_Data' data set
> table(All_Data$trainortest, All_Data$Street)

      Grvl Pave
test      1 364
train     5 1090
> Partitioning_Table <- table(All_Data$trainortest, All_Data$Street)
> prop.test(Partitioning_Table, correct = FALSE)$p.value
[1] 0.6366583
```

Question 4: Use the *rpart()* command to create a CART model using the Training data with the *over180* variable as the target and all other variables as predictors. Plot the model, using the *type* and *extra* options to improve the visualization.

```
> Train_Cart <- rpart(over180 ~., data = Train_Data, method = "class")
> rpart.plot(Train_Cart, type = 4, extra = 102)
> Pred_Train_Cart <- predict(object = Train_Cart, newdata = Train_Data, type = "class")
. . . . .
```

Question 5: Obtain the accuracy of the model based on the Training data and based on the Test data. Compare the measures of accuracy to discuss the model's performance.

```
> Pred_Train_Cart <- predict(object = Train_Cart, newdata = Train_Data, type = "class")
> table(Train_Data$over180, Pred_Train_Cart)

      Pred_Train_Cart
      Over $180K Under $180K
Over $180K      375      48
Under $180K      43     629
> 375 + 629
[1] 1004
> dim(Train_Data)[1]
[1] 1095
> 1004/1095
[1] 0.916895
```

```
> #Now we compare our Training cart model predictions with our testing data
> Pred_Test_Cart <- predict(object = Train_Cart, newdata = Test_Data, type = "class")
> table(Test_Data$over180, Pred_Test_Cart)

      Pred_Test_Cart
      Over $180K Under $180K
Over $180K      123      18
Under $180K      17     207
> 123 + 207 # = 330
[1] 330
> dim(Test_Data)[1] # = 365
[1] 365
> 330/365 # = 0.9041096
[1] 0.9041096
```

Question 6: Now use the *trainControl()* and *train()* commands to make a CART model using 10-fold cross-validation. Plot the model.

```
> set.seed(134)
> Train_Control <- trainControl(method = "cv", number = 10)
> Train_Data_CCA <- na.omit(Train_Data)
> model <- train(over180 ~., data = Train_Data_CCA, method = "rpart", trControl = Train_Control)
> print(model)

> plot(model$finalModel)
> text(model$finalModel)
> fancyRpartPlot(model$finalModel, cex = 1.2, main = "Cart Model for houses that sold for <$180")
```

Question 7 & 9: Obtain the accuracy of the model based on the Training data and based on the Test data. Compare the measures of accuracy to discuss the model's performance.

```
> pred_Kfold_model <- rpart.predict(object = model, newdata = Train_Data)
> table(Train_Data$over180, pred_Kfold_model)
      pred_Kfold_model
      Over $180K Under $180K
Over $180K      383         40
Under $180K      146        526
> 383+526 # = 909
[1] 909
> dim(Train_Data)[1] #1095
[1] 1095
> 909/1095 # = 0.830137
[1] 0.830137

> pred_kfold <- rpart.predict(object = model, newdata = Test_Data)
> table(Test_Data$over180, pred_kfold)
      pred_kfold
      Over $180K Under $180K
Over $180K      134         7
Under $180K       56        168
> 134 + 168 # = 302
[1] 302
> dim(Test_Data)[1] # = 365
[1] 365
> 302/365 # = 0.8273973
[1] 0.8273973
```